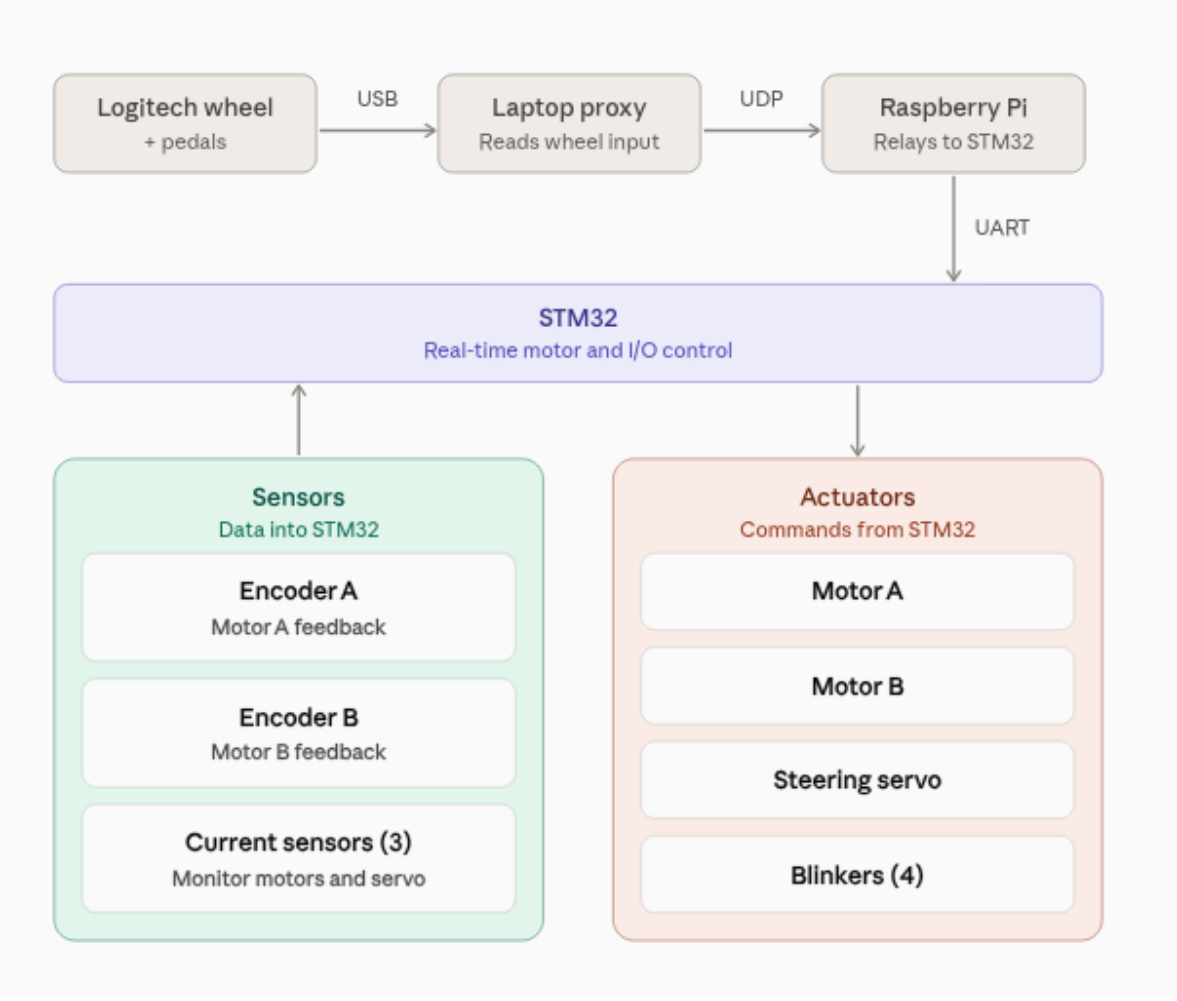
18-649 Lab 2: Report Document

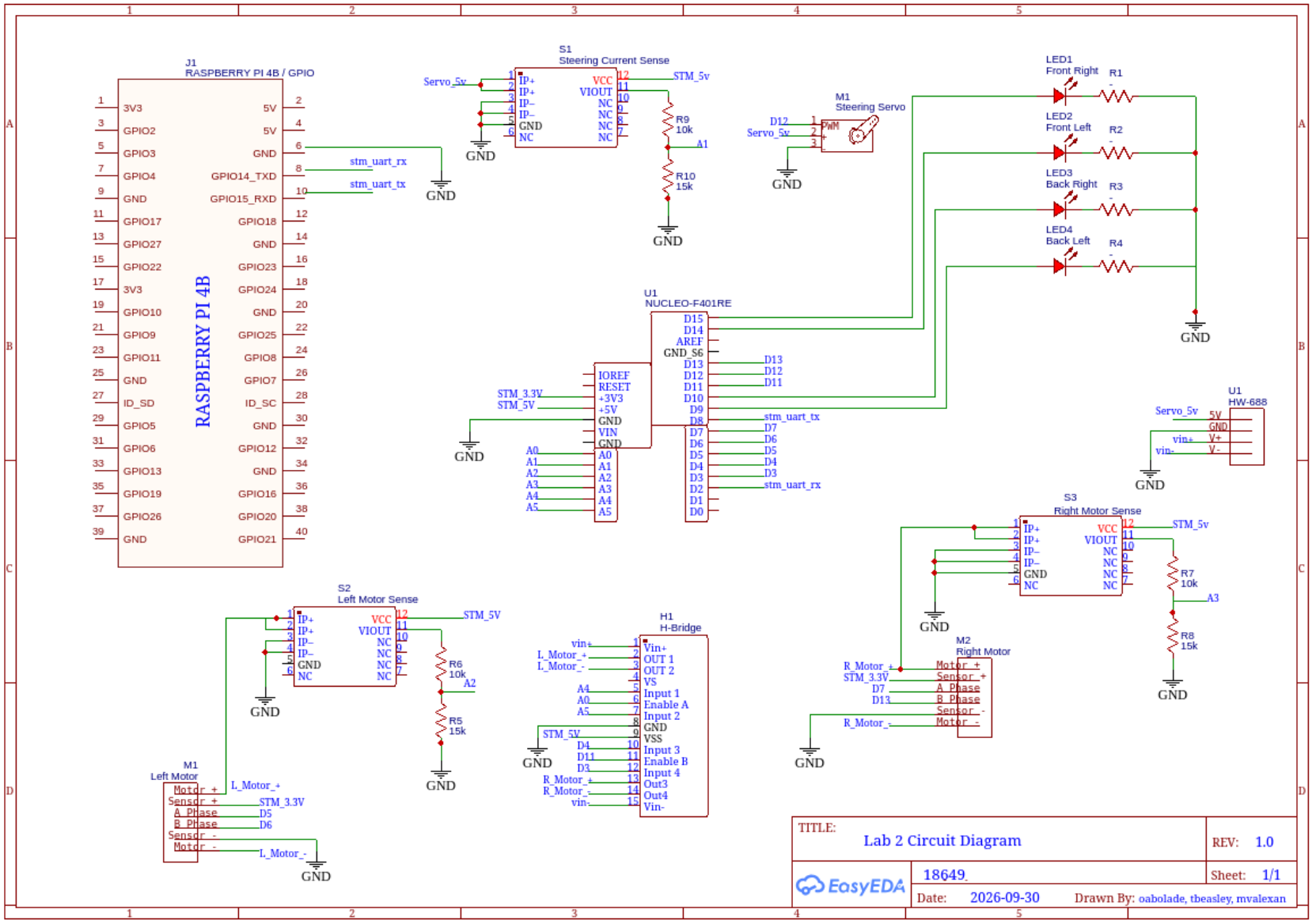
Misho Alexandrov (mvalexan), Ore Abolade (oabolade), T'sairus Beasley (tbeasley)

Repository: https://github.com/o-abolade/18-649-Tabletop-Drive-by-Wire-Car **Latest commit:** `4d2fe12`

System Diagram



Circuit Diagram



Team-Defined Values

Status guide: Fixed by handout = mandatory; Proposed = suggested behavior; Hardware-dependent = depends on purchased or developed hardware; Design-dependent = software/interface choice; Needs calibration = must be measured or tuned.

Driver inputs and steering

Value	Current choice / units	Reason / hardware dependency	Status
Steering input	Normalized command: −1000 to +1000; wheel raw range: −32768 to +32767	The Pi maps its wheel input to the normalized command before transmitting it to the STM32.	Implemented in Lab 2
Servo travel	20 ms period; left/center/right = 1000/1500/2000 µs (allowed range 500–2500 µs)	Values were verified with the front steering servos and retain margin from the electrical endpoints.	Implemented and tested
Steering mapping	Linear: −1000/0/+1000 → 1000/1500/2000 µs	Monotonic mapping across the tested usable steering range.	Implemented in Lab 2
Throttle input	Normalized command: 0 to 1000; raw pedal released/full: +32767/−32768	The Pi normalizes throttle and brake before transmission; remaining pedal-noise tuning is outside this Lab 2 value.	Implemented in Lab 2
Throttle mapping	Linear: 0–1000 command → 0–6500 encoder transitions/s target	The target is used by the per-motor PID when encoder feedback is valid.	Implemented; feedback calibration pending
Maximum speed	6500 encoder transitions/s target (about 90% of the 7227 transitions/s observed maximum)	Physical wheel speed in rpm or m/s still depends on verified encoder scale and wheel diameter.	Provisional Lab 2 value
Brake activation	Brake command ≥50 on the normalized 0–1000 scale	Small brake-input noise below this threshold does not engage braking.	Implemented in Lab 2
Brake priority	Brake overrides throttle	Required by S2; dynamic braking sets each H-bridge input pair high and holds its enable at 100%.	Fixed by handout

Turn signals, buttons and force feedback

Value	Current choice / units	Reason / hardware dependency	Status
Turn thresholds	Left ≤−400; right ≥+400 on the normalized steering scale	Thresholds are approximately 40% of the normalized steering range.	Implemented in Lab 2
Cancel hysteresis	None in Lab 2; cancel occurs after crossing the same threshold on the return stroke	No separate cancellation hysteresis was needed for the Lab 2 implementation.	Implemented in Lab 2
Repeated turn press	Same-side press toggles that signal off	A turn signal can also cancel after the completed steering maneuver.	Implemented in Lab 2
Both turn buttons	Ignore conflicting request; retain turn state	Team choice; error hazards still override.	Proposed
Held turn button	Act on press edge, not every UART command update	Avoid repeated actions while held.	Proposed
Test-button debounce	Presses <50 ms apart count as one	R2.5; button hardware still needs checking.	Fixed by handout
Restore window	Two presses within 500 ms	R3; applies to each zone's test input.	Fixed by handout
Force sensing	TBD: current sensing or position error	Depends on servo, available sensors and PCB.	Hardware-dependent

Value	Current choice / units	Reason / hardware dependency	Status
Force mapping	TBD: zero, sign, gain, deadband	Depends on measured load and wheel response.	Needs calibration
Force output limit	-100 to 100; signed value	Front Zone description and UDP encoding.	Fixed by handout

Drivetrain, encoders and motor limits

Value	Current choice / units	Reason / hardware dependency	Status
Encoder scale	TBD: counts per wheel revolution	Encoder resolution, decoding mode and gearing.	Hardware-dependent
Wheel size	TBD; diameter in metres	Needed if speed is expressed in m/s.	Hardware-dependent
Speed feedback	Per-motor quadrature transition rate measured over the control period	Each rear motor has an independent PID; encoder signal reliability still requires hardware validation.	Implemented; validation pending
Speed-control period	10 ms	The firmware updates commands and speed control at this period.	Implemented in Lab 2
Control gains	PID: Kp = 0.02, Ki = 0.01, Kd = 0.00	Initial values in firmware; tune only after repeatable encoder feedback is verified.	Provisional Lab 2 values
Maximum PWM	100%; minimum effective nonzero drive duty = 55%	The 55% floor avoids commands below the observed motor-drive threshold; thermal/current limits remain uncalibrated.	Implemented; limits pending
Stuck-wheel trigger	Nonzero command; no encoder edges ≥200 ms	Rear Zone description.	Fixed by handout
Stuck-wheel increase	TBD: duty step/ramp and update interval	Increase only to chosen PWM cap; check heating.	Needs calibration
Motor current limits	No ampere thresholds in Lab 2; raw 12-bit ADC reads are available on A2 (left), A3 (right), and A1 (steering)	Planned divider per 5 V sensor: 10 kΩ top and 14.7 kΩ bottom (10 kΩ + 4.7 kΩ), yielding a 0.595 ratio and 2.98 V at 5 V input. Sensor calibration is still required.	ADC mapping implemented; thresholds TBD
Current-fault filtering	TBD: persistence time; ms	Reject normal transients; meet fault response needs.	Needs calibration

CAN communication and fault handling

Inherited-table note: These CAN and zone-fault values were not implemented or revised during Lab 2; they remain as inherited planning entries.

Value	Current choice / units	Reason / hardware dependency	Status
CAN identifiers	Proposed in the CAN message catalog; final assignment TBD	Assign priorities; brake/error before convenience.	Design-dependent
Payload definitions	Proposed in the CAN message catalog; final scaling and valid ranges TBD	Document in CAN message catalog; sensor ranges matter.	Design-dependent
Sequence counter	At least 4 bits; final width TBD	R4.4; define wraparound and duplicate tracking.	Fixed minimum; design TBD
CAN bit timing	500 kbit/s; timing segments TBD	R4.2; depends on chosen controllers and clocks.	Fixed rate; hardware TBD
Control-message cadence	Every UDP update; gap ≤50 ms	R4.4; receiving hardware/software must support it.	Fixed by handout

Value	Current choice / units	Reason / hardware dependency	Status
Health indication	TBD: LED colors and on/off meaning	Depends on PCB LEDs; map healthy/degraded states.	Design-dependent
Watchdog timeout	TBD; ms	STM32 watchdog clock, task timing and fault handling.	Hardware/design-dependent

Task Table

Name	Type	Period	Priority	Deadline	Talks to
UART RX ISR (`uart_isr`)	ISR	Event-driven, per byte	Hardware interrupt (preempts all threads)	Drain each byte before the next arrives (~87µs @ 115200 baud)	USART1 peripheral; writes `latest_cmd` (spinlock), restarts byte-timeout timer, gives `cmd_received_sem`
Byte-timeout timer (`byte_timeout_expired`)	`k_timer` callback	One-shot, restarted on every byte; fires after 20ms of silence	Timer/workqueue context	Fire within 20ms to force parser resync	Resets `rx_state`/`rx_idx` in `pi_stm32_uart.c`
Encoder edge ISRs (`phase_a_callback` / `phase_b_callback`, ×2 wheels)	ISR	Event-driven, per quadrature edge	Hardware interrupt	Process each edge before the next arrives at max wheel RPM	GPIO interrupt lines; updates atomic `count`/edge counters in `encoder.c`
Control thread (`control_thread_fn`)	Thread	Event-driven (wakes on `cmd_received_sem`), with a 10ms fallback timeout	5	2ms (R2.1/R2.2 throttle/brake), 50ms (R2.3 steering)	Reads latest command via `pi_stm32_uart`; drives `motor_control`, `steering`, `blinker_ctrl`; runs PID via `apply_throttle`
Blinker thread (`blinker_thread_fn`)	Thread	Variable: 500ms (normal, 1Hz) / 250ms (hazard, 2Hz), toggling each half-period	6	100ms to start blinking (R2.4); blink rate ±10% of nominal	`blinker.c` GPIOs; reads `blink_state`/`past_threshold` (mutex-protected)
Status/heartbeat thread (`status_thread_fn`)	Thread	20ms	7	20ms ±10% heartbeat cadence	Reads `current` sensors (ADC); sends status frame via `pi_stm32_uart`
Main (`main()`)	Thread, runs once at boot	N/A	0 (default)	N/A	Initializes every module once at startup
Bench/shell commands (`motor_pulse`,	Thread (shell), manually triggered	Aperiodic	Shell thread priority	N/A — bench-only, not part of runtime control loop	Directly drives `motor_control` /

Name	Type	Period	Priority	Deadline	Talks to
`steering_test`, `encoder_monitor`, etc.)					`steering` / `encoder` on demand

Integration Writeup

In the final design, the Raspberry Pi continues to perform the same functions as in Lab 2. It receives the UDP wheel stream from the laptop proxy, decodes it, sends commands to the microcontroller at least every 50 ms, and returns a status frame every 20 ms. The single STM32 is divided into two zones connected by a physical CAN bus. The steering zone takes over the steering servo, the servo current sensor, and the front blinkers (FL and FR). The drivetrain zone takes over motor driving, the motor current sensors, the encoders, and the rear blinkers (RL and RR). We assume the Pi's UART terminates on the steering zone, which acts as a gateway: it translates Pi commands into CAN frames and collects CAN status data for the Pi's status frame.

Messages and Arbitration

Each in-memory variable in the Lab 2 firmware that must cross a zone boundary becomes a CAN message, as summarized in Table 1. The identifiers shown are proposed values.

Table 1. Proposed CAN message catalog

Message	Proposed ID	Sender → receiver(s)	Payload layout	Rate / trigger	Priority reason
SYSTEM_ERROR	0x080	Any zone → all zones	B0: version + seq[3:0]; B1: source zone; B2: error reason; B3: state; B4–B7: reserved	On SYSTEM_FAIL entry; repeat ≤50 ms while active	Highest: commands safe outputs.
VEHICLE_CONTROL	0x100	Cockpit → steering, drivetrain	B0: version + seq[3:0]; B1: throttle 0–100%; B2: brake 0–100%; B3–B4: steering −1000…+1000; B5: turn state; B6: control flags; B7: reserved	Every UDP update; gap ≤50 ms	High: carries brake and current driver control.
HEARTBEAT_COCKPIT	0x300	Cockpit → steering, drivetrain	B0: version + seq[3:0]; B1: INIT/NORMAL/LOCAL_FAIL/SYSTEM_FAIL; B2: CAN A/B health flags; B3: error code; B4–B7: reserved	Every 20 ms ±10% while the zone is healthy	Needed for failure detection.
HEARTBEAT_STEERING	0x301	Steering → cockpit, drivetrain	Same layout as HEARTBEAT_COCKPIT	Every 20 ms ±10% while the zone is healthy	Needed for failure detection.
HEARTBEAT_DRIVETRAIN	0x302	Drivetrain → cockpit, steering	Same layout as HEARTBEAT_COCKPIT	Every 20 ms ±10% while the zone is healthy	Needed for failure detection.
FORCE_FEEDBACK	0x400	Steering → cockpit	B0: version + seq[3:0]; B1: signed force −100…+100; B2: load status; B3–B7: reserved	At least every 50 ms	Convenience feedback; below safety messages.

Because CAN arbitration favors the lower identifier, identifier order doubles as priority order. The error-state message is assigned the lowest ID, followed by brake, steering, throttle, and blinker messages, with status and heartbeat frames last. This ordering reflects the deadlines in our requirements: the brake path has a 2 ms budget, steering has 50 ms, and blinkers have 100 ms. A brake command should never wait behind any lower-priority frame other than one already in transmission.

Redundant Buses

The system uses two CAN buses that carry identical traffic. Every message is transmitted on both buses, so each controller receives every frame twice and must discard the duplicate. Each frame carries a sequence counter; a controller accepts the first copy it observes and drops the second, which ensures that a command is never applied twice and that either bus alone is sufficient to operate the vehicle.

Bus Health Monitoring

Each controller must also monitor the health of both buses and detect degradation, not only total loss. For each bus, a controller tracks the time since the last received frame relative to the expected message rate, whether frames are arriving on one bus but not the other, and the CAN error counters and bus-off events. A bus that falls behind its counterpart is marked degraded, and this status is included in the status frame so that it is visible to the Pi. If both buses fall silent, or a zone stops sending heartbeats, the system enters the error state: the drivetrain zone brakes the motors, the steering zone holds position, and all four blinkers flash together as hazards.

Blinker Synchronization

In Lab 2, the front and rear blinkers shared a single MCU and time base. After the split, each half runs on a separate MCU with its own oscillator, so independent 1 Hz timers will gradually drift apart and may violate the 1 ms front/rear synchronization requirement. One zone must therefore own the blink timebase and broadcast BLINK_SYNC, and the other zone must correct its local timer when each message arrives. Because bus latency varies with arbitration, this correction carries a jitter bound that we will need to measure. If synchronization messages stop arriving, each zone continues to free-run and flags the loss.